Chapter 14: Currency Converter

Overview A Currency Converter is a practical application that allows users to convert one currency into another based on exchange rates. This project enhances understanding of API integration, user input handling, and mathematical calculations in Python.

This chapter covers the step-by-step implementation of a Currency Converter, fetching real-time exchange rates, performing currency conversions, and displaying the results.

Key Concepts of Currency Converter in Python

- **Exchange Rates Handling:**
  - Using real-time exchange rates via an API (e.g., exchangeratesapi.io , Open Exchange Rates)
  - Using predefined exchange rates for offline mode
- **User Input Handling:**
  - Accepting user input for source and target currency
  - Validating correct currency codes
- **Arithmetic Operations:**
  - Multiplication for currency conversion (converted_amount = amount * exchange_rate)

Currency Conversion Table

From	To	Conversion Rate (Example)
USD	EUR	0.85
EUR	GBP	0.86
GBP	INR	101.56
INR	USD	0.012
USD	JPY	110.25

Basic Rules for Currency Converter in Python

Rule	Correct Example
Use an API for real-time rates	requests.get("https://api.exchangerate-api.com/v4/latest/USD")
Validate user input as a currency code	if currency in exchange_rates:
Perform conversion using rates	converted_amount = amount * exchange_rates[target_currency]

Syntax Table

SL	Concept	Syntax/Example	Description
1	Fetch exchange rates	requests.get("API_URL")	Fetches live currency rates from an API
2	Get user input	amount = float(input("Enter amount: "))	Takes input for currency amount
3	Validate input	if currency in exchange_rates:	Ensures input is a valid currency code
4	Perform conversion	converted = amount * exchange_rates[to_currency]	Multiplies the amount by the exchange rate
5	Display result	print(f"Converted amount: {converted:.2f}")	Prints the converted

				currency value

Real-Life Project: Currency Converter

Project Code:

```
1.  import requests

2.  def get_exchange_rates(base_currency):
3.      url = f"https://api.exchangerate-api.com/v4/latest/{base_currency}"
4.      response = requests.get(url)
5.      data = response.json()
6.      return data['rates']

7.  base_currency = input("Enter base currency (e.g., USD, EUR): ").upper()
8.  target_currency = input("Enter target currency: ").upper()
9.  amount = float(input("Enter amount: "))

10. rates = get_exchange_rates(base_currency)

11. if target_currency in rates:
12.     converted_amount = amount * rates[target_currency]
13.     print(f"{amount} {base_currency} is {converted_amount:.2f} {target_currency}")
14. else:
15.     print("Invalid currency code!")
```

Project Code Explanation Table

Line	Code Section	Description
1	import requests	Imports the requests module to fetch exchange rates from

		an API.
2-6	def get_exchange_rates(base_currency):	Defines a function to fetch exchange rates for a given base currency.
3-4	url = f"https://api.exchangerate-api.com/v4/latest/{base_currency}"	Constructs the API URL using the base currency.
5	data = response.json()	Parses the response as JSON.
7	base_currency = input("Enter base currency: ").upper()	Takes the base currency input from the user.
8	target_currency = input("Enter target currency: ").upper()	Takes the target currency input from the user.
9	amount = float(input("Enter amount: "))	Takes the amount for conversion and converts it to a float.
10	rates = get_exchange_rates(base_currency)	Calls the function to get

		exchange rates.
11 - 12	if target_currency in rates:	Checks if the target currency is valid and performs conversion.
13	print(f"{amount} {base_currency} is {converted_amount:.2f} {target_currency}")	Prints the converted currency amount.
14 - 15	else:	Handles invalid currency codes by displaying an error message.

Expected Results

- The program asks the user for base and target currency codes.
- It fetches exchange rates from an API.
- The user enters an amount to convert.
- The program calculates and displays the converted amount.
- If an invalid currency code is entered, an error message is displayed.

Hands-On Exercise Try improving the Currency Converter with these additional features:

1. **Allow offline mode** with predefined exchange rates.

2. **Support multi-currency conversions** in one execution.
3. **Cache exchange rates** to reduce API requests.
4. **Use GUI (Tkinter) for better user experience.**
5. **Allow conversion history tracking** and display past conversions.

Conclusion This Currency Converter project introduces Python concepts such as API integration, user input handling, and arithmetic operations. By expanding this project, developers can create a more advanced financial tool with real-time exchange rate tracking.